

Claude Code Avançado: quatro frentes de automação

Loop, SubAgents, Agent Teams e Hooks — quatro recursos avançados implementados, testados e documentados nas sessões deste módulo.

`github.com/fernandojsmelo/FormacaoClaudeCode2026` · Consolidado a partir dos relatórios de sessão e material das aulas

- Loop
- SubAgents
- Agent Teams
- Hooks

O que foi coberto neste módulo

Este documento reúne, em formato executivo, os quatro recursos avançados do Claude Code trabalhados ao longo das sessões deste módulo — cada um com material teórico e um relatório de sessão prático já salvos nas respectivas pastas. Aqui você encontra o essencial de cada um lado a lado, mais uma tabela comparativa para decidir rapidamente qual recurso usar em cada situação.

- 01** Loop — Agendamento nativo do Claude Code: repete um prompt ou slash command em intervalo fixo ou dinâmico, enquanto a sessão do terminal estiver aberta.
- 02** SubAgents — Agente auxiliar com contexto isolado e tools restritas, invocado pelo agente principal para executar uma tarefa focada e devolver só o resultado final.
- 03** Agent Teams — Vários sub-agents especializados rodando em paralelo (fan-out) sobre o mesmo problema; o orquestrador aguarda todos e sintetiza um relatório único (fan-in).
- 04** Hooks — Scripts que interceptam eventos do ciclo de vida do Claude Code (antes/depois de uma tool, ao notificar, ao encerrar) para automação determinística.

Agendamento nativo de tarefas repetitivas

Roda um prompt ou slash command repetidamente na mesma sessão do Claude Code, em intervalo fixo (--interval/--cron) ou dinâmico — quando o intervalo é omitido, o próprio modelo decide quando reagendar a próxima rodada, conforme o que encontrou. O comando irmão /schedule cobre o caso de execução única num horário futuro, sem repetição.

SINTAXE USADA NA PRÁTICA

```
/loop [intervalo] <prompt ou comando>
```

PERSISTÊNCIA

Só enquanto o terminal está aberto — não é cron do SO

PARA SOBREVIVER AO FECHAMENTO

Skill schedule (agentes com cron real na nuvem)

TESTADO NESTA SESSÃO

3 slash commands, modo headless, repo temporário isolado

```
/loop 20m /verificar-novos-materiais    # cronograma fixo
/loop /checar-padrao-pastas             # modo dinâmico
/schedule "next monday 09:00" "npm audit" # execução única
```

- Os 3 exemplos testados (commit automático, edição cirúrgica de índice, checagem só-leitura) foram validados duas vezes cada — com e sem novidade — porque idempotência é pré-requisito para rodar sem supervisão.
- allowed-tools restrito por comando é rede de segurança real: o de commit não tem push, o de checagem só tem find/Read.
- Combina bem com Hooks: o /loop detecta o problema, um hook Notification comunica (ex.: só alerta no Slack se for crítico).

Especialistas com contexto isolado

Um agente auxiliar com sua própria janela de contexto, seu próprio system prompt e, opcionalmente, uma lista branca de tools — definido em Markdown com frontmatter YAML em `.claude/agents/` (local, versionado) ou `~/.claude/agents/` (global, pessoal; local tem prioridade em conflito de nomes).

INVOCAÇÃO

Automática (via description) ou explícita com `@nome-do-agente`

ISOLAMENTO

Contexto 100% separado — só o resultado final volta à conversa principal

TOOL RESTRICTIONS

Lista branca, até no nível de argumento:
`Bash(command:"npm test")`

RECARREGAMENTO

Não automático — mudanças só valem a partir da próxima sessão

```
---
name: code-reviewer
tools: Read, Grep, Glob, Bash
model: sonnet
---
```

- Exemplo 1 testado: `code-reviewer.md` revisou um arquivo com 4 bugs plantados e encontrou os 4, com a severidade correta e sem falso positivo — sem conseguir editar nada, pois tools não inclui `Write`.
- Exemplo 2 testado: paralelismo com agentes embutidos (`Explore`, `general-purpose`) — funciona disparando tarefas independentes no mesmo bloco de tool calls; não depende de subagentes customizados.
- Sem escalada de privilégio: um subagente nunca tem mais acesso do que o principal teria na mesma sessão; e a saída de um subagente é sempre tratada como dado, nunca como comando.

Orquestração paralela, Fan-Out/Fan-In

Vários sub-agents especializados trabalham em paralelo sobre o mesmo problema. O orquestrador dispara todos via Task tool (na mesma resposta, não em sequência), aguarda os que faltam e sintetiza um relatório único. É o padrão Fan-Out/Fan-In — o mais comum quando a tarefa tem dimensões independentes (segurança, qualidade, testes...).

PADRÃO USADO Fan-Out / Fan-In (demais: Pipeline sequencial, Revisão cruzada)	TOOL DO ORQUESTRADOR Task (Claude Code) · tool "Agent" equivalente no Claude Agent SDK
EXECUÇÃO REAL VALIDADA 3 sub-agents em paralelo · custo \$0.3758 · 1 turno	TAMBÉM DISPONÍVEL EM CÓDIGO Claude Agent SDK (Python) — AgentDefinition + query()

```
- Task(security-agent, diff)
- Task(quality-agent, diff)
- Task(test-agent, diff)
# disparados juntos na mesma resposta -> fan-out
# orquestrador aguarda os 3 -> fan-in (síntese unica)
```

- Time recomendado e testado: orchestrator.md + security-agent, quality-agent e test-agent, revisando exemplo/auth.js (segredo hardcoded, SQL injection, nomes ruins, zero testes) — todos os achados reais.
- Sub-agents não se comunicam entre si — a síntese final é responsabilidade exclusiva do orquestrador.
- Armadilhas conhecidas: escopos sobrepostos custam tokens em dobro; times acima de 5 agentes ficam caros e difíceis de sintetizar.

Automação determinística

Scripts shell disparados automaticamente em eventos do ciclo de vida do Claude Code — PreToolUse (antes, pode bloquear), PostToolUse (depois, não bloqueia), Notification e Stop — configurados em settings.json. Diferença essencial: uma Skill sugere (probabilístico); um Hook garante (determinístico).

CONFIG

hooks em settings.json — chave por tipo, matcher + command

BLOQUEAR/PERMITIR

PreToolUse: exit 2 bloqueia, exit 0 permite

TESTADO NESTA SESSÃO

Stop (som ao final) + PreToolUse (block-dangerous.sh)

TAMBÉM DISPONÍVEL

type: "http" — envia o evento a um servidor externo (timeout padrão 10s)

```
$ rm -rf /  
PreToolUse:Bash hook error: [bash .claude/hooks/block-dangerous.sh]:  
Bloqueado pelo hook PreToolUse: comando potencialmente destrutivo detectado (rm -rf /)
```

- Bateria de testes real: bloqueou `rm -rf /` e `git push --force` em main/master; permitiu `ls -la` e `rm -rf node_modules` — validado tanto isolado quanto disparando o comando de verdade no Claude Code.
- Limitação conhecida: o filtro é regex sobre a string do comando e pode ser contornado por ofuscação — funciona como rede de segurança adicional, não como sandboxing completo.
- Próximos passos sugeridos no relatório: cobrir `curl` | `bash`, `sudo` e `chmod 777`; adicionar PostToolUse de auto-formatação.

Comparativo rápido

RECURSO	O QUE RESOLVE	QUANDO USAR	ISOLAMENTO?	COMO ATIVAR
Loop	Repetir uma tarefa em intervalos, dentro da sessão aberta	Monitoramento/manutenção recorrente enquanto você trabalha	Não — mesma sessão, mesmo contexto	/loop [intervalo] <prompt>
SubAgents	Delegar uma tarefa especializada com tools restritas	Uma tarefa focada que pede permissões mínimas (revisão, auditoria)	Sim — contexto isolado; só o resultado final volta	Arquivo .md em .claude/agents/
Agent Teams	Rodar várias análises independentes ao mesmo tempo	Dimensões independentes do mesmo problema (segurança + qualidade + testes)	Sim — cada sub-agent isolado; orquestrador sintetiza	orchestrator.md + Task tool
Hooks	Garantir que algo sempre (ou nunca) aconteça	Validação/segurança/log que não pode depender do modelo decidir	Não se aplica — script determinístico, fora do modelo	hooks em settings.json

QUANDO USAR

Quando usar cada um

- **Loop** — uma tarefa recorrente que você quer ver rodando enquanto trabalha, sem sair da sessão atual (monitoramento, checagens periódicas).
- **SubAgents** — uma tarefa focada e sensível a permissões, que se beneficia de um contexto limpo e de tools restritas (revisão de código, auditoria pontual).
- **Agent Teams** — o mesmo problema tem várias facetas independentes que podem ser analisadas ao mesmo tempo, com um relatório único no final.
- **Hooks** — algo precisa sempre (ou nunca) acontecer, e você não pode depender do modelo lembrar de seguir a regra (segurança, auditoria, notificação).

DOCUMENTOS COMPLETOS DO MÓDULO

Claude Code Avancado/Loop/relatorio-sessao-loop.pdf

Claude Code Avancado/Loop/Compreendendo o Comando loop/Loop_Tarefas_Agendadas.pdf

Claude Code Avancado/SubAgents/relatorio-subagents-na-pratica.pdf

Claude Code Avancado/SubAgents/Compreendendo SubAgents/SubAgentsContextoIsolado.pdf

Claude Code Avancado/Agent Teams/Agent_Teams_Relatorio_Sessao.pdf

Claude Code Avancado/Agent Teams/Compreendendo sobre Agent Teams/Agent_Teams_Orquestracao_Paralela.pdf

Claude Code Avancado/Hooks/relatorio_hooks.pdf

Claude Code Avancado/Hooks/Compreendendo os Hooks/Hooks_Automacao_Deterministica.pdf

Fontes: relatórios de sessão e material das aulas do módulo Claude Code Avançado (repositório FormacaoClaudeCode2026).

Este resumo consolida o que foi de fato implementado, testado e documentado nas sessões de trabalho — nenhum conteúdo foi inventado além do que consta nos PDFs de origem.